

Adaptive k -Allocation for k -MIP Attention in Graph Transformers: A Compute-Constrained Empirical Study

Abstract

The k -Maximum Inner Product (k -MIP) attention enhances the scalability of graph transformers by limiting each query to its top- k keys. However, the original approach keeps a fixed k throughout training and all model layers [1]. This report explores whether adapting k over time can reduce supervision bottlenecks and boost real-world performance. Using GraphGPS, we compare four main strategies, a fixed k , global cosine annealing, layerwise cosine annealing, and a hybrid layerwise+entropy-adaptive method. Our study, constrained by computation, involves three random seeds, 50 epochs on two peptide benchmarks, and 60 epochs on PascalVOC-SP. Results vary significantly by dataset. Fixed $k = 15$ performs best on Peptides-func (63.01 ± 0.67 AP) and Peptides-struct (0.2652 ± 0.0039 MAE), while adaptive methods shine on PascalVOC-SP: the hybrid approach achieves 50.35 ± 5.66 F1 versus 47.23 ± 7.53 for fixed k . Analyzing these strategies as a component ablation reveals that temporal, depthwise, and querywise adaptation do not always provide universal benefits. Instead, such gains are observed mainly on the larger PascalVOC-SP node classification task. Entropy trajectories and per-layer k profiles show that the schedules function as intended, even when predictive performance stays static, emphasizing a distinction between control mechanisms and actual advantages. Runtime and memory measurements further underline practical considerations: adaptive schedules often alter total training time due to differences in stopping criteria and tend to increase memory consumption. Ultimately, this work offers a nuanced outcome: adaptive sparsification isn't consistently superior across all LRGB tasks, but it can be advantageous on larger, more complex graphs when appropriately tuned.

1 Introduction

Graph transformers combine the local inductive bias of message-passing neural networks (MPNNs) with the global receptive field of self-attention, making them attractive for graph tasks that require long-range interactions [2, 3]. However, their main drawback is the quadratic cost of dense all-pairs attention. This cost becomes particularly restrictive on large graphs, where even moderate hidden dimensions make dense $N \times N$ attention impractical.

k -MIP attention and GraphGPS. De Schouwer et al. [1] address this bottleneck with k -Maximum Inner Product (k -MIP) attention, which restricts each query to its top- k keys before the softmax. Given node features $\mathbf{X} \in \mathbb{R}^{N \times d}$ and learnable projections $\mathbf{W}_Q^h, \mathbf{W}_K^h \in \mathbb{R}^{d \times d_K}$ and $\mathbf{W}_V^h \in \mathbb{R}^{d \times d_V}$ for head h , the query, key, and value matrices are $\mathbf{Q}^h = \mathbf{X}\mathbf{W}_Q^h$, $\mathbf{K}^h = \mathbf{X}\mathbf{W}_K^h$, $\mathbf{V}^h = \mathbf{X}\mathbf{W}_V^h$. The dense score matrix is $\mathbf{S}^h = \frac{1}{\sqrt{d_K}}\mathbf{Q}^h(\mathbf{K}^h)^\top$, and k -MIP keeps only the top- k entries in each row before the softmax

$$\tilde{S}_{ij}^h = \begin{cases} S_{ij}^h, & j \in \text{TopK}_i(k), \\ -\infty, & \text{otherwise.} \end{cases}$$

The attention weights become $A_{ij}^h = \frac{\exp(\tilde{S}_{ij}^h)}{\sum_{m=1}^N \exp(\tilde{S}_{im}^h)}$, and the head output is $\mathbf{A}^h \mathbf{V}^h$. In practice, the top- k search is implemented lazily with KeOps to avoid materialising the full score matrix in memory [4].

Integrated into GraphGPS [2], which stacks local message passing (GatedGCN), global attention, and feedforward blocks with residual connections, k -MIP attention yields linear memory complexity while preserving strong empirical performance. The original paper also proves that k -MIP transformers can approximate full-attention transformers arbitrarily well and analyses the expressive power of the broader GraphGPS framework [1].

A central limitation highlighted by the authors is *supervision starvation* meaning, keys that never enter the top- k set receive no gradient signal through the attention weights, making it difficult for initially low-scoring but potentially useful keys to become competitive [1]. This creates a self-reinforcing exclusion where early mistakes in the score ranking can persist because excluded keys never get a chance to become competitive. A natural remedy suggested by the paper is to start with a larger attention budget and reduce it over training, turning k into a curriculum-like control variable rather than a fixed constant [5].

Research question. This report investigates whether adaptive k -allocation can mitigate supervision starvation in k -MIP attention and enhance predictive accuracy compared to a fixed k . A fixed k treats all layers, queries, and training phases as equally sparse, which may not be optimal. In contrast, adaptive schedules can potentially leverage the early exploratory benefits of broader attention while preserving the efficiency of sparse attention later.

Our hypotheses suggest that such adaptive schedules may be especially beneficial for larger or more diverse graphs, and that depth-aware allocation may be more stable than querywise adaptation alone.

Goals. To address this question, this report sets out four main objectives,

- To develop a unified scheduling framework for temporal, layerwise, and querywise k -allocation.
- To evaluate these schedules on three LRGB datasets using the same GraphGPS+ k -MIP backbone.
- To interpret the four methods as elements of a component ablation involving temporal, depth-wise, and entropy-based adaptation.
- To compare predictive performance, runtime, and memory usage within a fixed compute budget (24 GPU hours on a single A100).

2 Method: Adaptive k -Allocation

We decompose the k budget into three orthogonal dimensions, temporal allocation across epochs, spatial allocation across depth, and dynamic allocation across queries.

2.1 Temporal schedule

Let $k_{\max}$ and $k_{\min}$ denote the schedule endpoints. After an optional warmup of t_w epochs, the base budget follows a cosine decay where

$$k_{\text{base}}(t) = k_{\min} + \frac{1}{2}(k_{\max} - k_{\min}) \left(1 + \cos\left(\pi \frac{t-t_w}{T-t_w}\right) \right), \quad (1)$$

for $t \geq t_w$, where T is the total number of epochs. During warmup, $k_{\text{base}}(t) = k_{\max}$. Fixed- k attention is recovered by setting $k_{\max} = k_{\min} = k$.

2.2 Layerwise allocation

We assign a layer-specific budget by scaling the base budget with a linear decay over depth

$$k_l(t) = \max\left(k_{\min}, \lfloor k_{\text{base}}(t) s_l \rfloor\right), \quad s_l = 1 - (1 - \rho) \frac{l}{L-1}, \quad (2)$$

where $\rho = 0.45$ is the deepest-layer ratio. The final layer therefore receives 45% of the shallowest-layer budget. This operationalises the intuition that lower layers need broader attention to discover patterns, whereas upper layers can refine information more selectively.

2.3 Entropy-adaptive per-query k

Within each layer budget $k_l(t)$, the hybrid method adapts the effective number of keys per query according to the entropy of the current attention distribution. For query i , the normalised entropy is

$$\bar{H}_i = \left(\frac{H(\mathbf{a}_i)}{\log k_l(t)} \right)^\alpha, \quad H(\mathbf{a}_i) = - \sum_j a_{ij} \log a_{ij}, \quad (3)$$

with $\alpha = 1.25$. The per-query budget is then

$$k_i = k_{\min} + \lfloor (k_l(t) - k_{\min}) \bar{H}_i \rfloor. \quad (4)$$

High-entropy queries receive more keys, while already-concentrated queries are pushed toward the minimum budget. Substituting Eq. (3) into Eq. (4) makes the intended behaviour explicit i.e., k_i increases monotonically with uncertainty and collapses back toward $k_{\min}$ when the attention mass is already sharply peaked.

2.4 Choosing the k range

To reason about schedule endpoints, we express them relative to graph size. Let $\bar{N}$ be the mean number of nodes in a dataset, we define the coverage ratios

$$\gamma_{\max} = \frac{k_{\max}}{\bar{N}}, \quad \gamma_{\min} = \frac{k_{\min}}{\bar{N}}. \quad (5)$$

These ratios indicate roughly what fraction of the graph each query can inspect at the start and end of training. In our final settings, the Peptides datasets use $45 \rightarrow 5$ with $\bar{N} \approx 152$, giving $\gamma_{\max} \approx 0.30$ and $\gamma_{\min} \approx 0.03$. PascalVOC-SP uses $30 \rightarrow 10$ with $\bar{N} \approx 400$, giving $\gamma_{\max} \approx 0.075$ and $\gamma_{\min} \approx 0.025$.

These choices were not arbitrary. On the smaller peptide graphs, an initial budget of 45 is still computationally manageable and gives each query a broad exploratory window, while a final budget of 5 keeps the tail sparse without collapsing to near-zero coverage. On PascalVOC-SP, reusing $45 \rightarrow 5$ would have driven the final coverage to about 1.25% of the graph, which exploratory runs suggested was too aggressive. The final $30 \rightarrow 10$ schedule was chosen to keep the terminal budget closer to 2–3% coverage while still giving a clearly larger early-training budget than the fixed baseline. A cautious rule of thumb from this study is therefore to choose $k_{\min}$ so that end-of-training coverage remains at a few percent of the average graph size, and then choose $k_{\max}$ to be several times larger without becoming effectively dense.

2.5 Datasets, empirical setup, and assumptions

We evaluate on three Long Range Graph Benchmark datasets [6]. **Peptides-func** contains 15,535 molecular graphs with about 152 nodes on average and is evaluated by average precision (AP, higher is better). **Peptides-struct** uses the same graphs for multi-target regression and is evaluated by mean absolute error (MAE, lower is better). **PascalVOC-SP** is a superpixel node-classification benchmark with 21 classes and is evaluated by weighted F1 (higher is better).

All models use the same GraphGPS+ k -MIP backbone with $L = 8$ layers, $H = 4$ attention heads, hidden dimension $d = 52$ (about 314k parameters), GatedGCN local message passing, and a two-layer output MLP. Training uses AdamW with cosine learning-rate decay [7]. Due to compute limits, the final study uses **50 epochs** for the two peptide datasets and **60 epochs** for PascalVOC-SP, while keeping the within-dataset configuration fixed across methods so that each comparison is fair inside a dataset. The Peptides tasks use learning rate 10^{-3} , batch size 32, patience 12, and 5-epoch learning-rate warmup with 3 epochs of k warmup. PascalVOC-SP uses learning rate 5×10^{-4} , batch size 8, patience 25, and 10-epoch warmup for both learning rate and k . All main results are reported over **three seeds**, and runtime plus peak-memory statistics are averaged over the same runs. The history-based diagnostic figures (training curves, entropy evolution, and layerwise k profiles) are shown from representative runs because they require full per-epoch traces rather than seed-aggregated summaries.

The four compared strategies are:

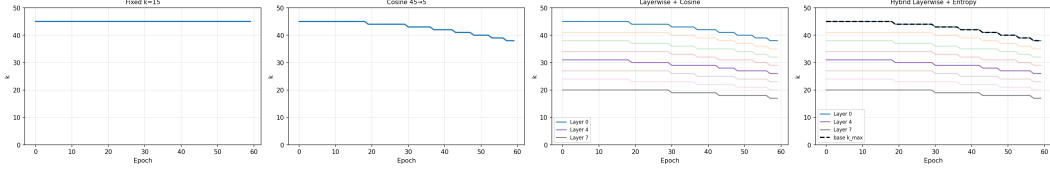


Figure 1: The four k -allocation strategies used in the study. From left to right: fixed k , global cosine annealing, layerwise cosine annealing, and hybrid layerwise+entropy adaptation. The layerwise and hybrid variants do not simply lower k over time; they also redistribute the attention budget across depth.

Table 1: Three-seed test performance (mean $\pm$ std). Best result per dataset in **bold**. The paper row is included for context and is not directly comparable because our parameter budget is smaller and PascalVOC-SP uses a custom split.

Method	Pept-func AP $\uparrow$	Pept-struct MAE $\downarrow$	PascalVOC-SP F1 $\uparrow$
Paper GPS+ k -MIP [1]	66.27	0.2562	39.69
Fixed $k = 15$	63.01 $\pm$ 0.67	0.2652 $\pm$ 0.0039	47.23 $\pm$ 7.53
Cosine annealing	61.14 $\pm$ 1.67	0.2784 $\pm$ 0.0056	47.99 $\pm$ 0.97
Layerwise cosine	60.77 $\pm$ 0.74	0.2812 $\pm$ 0.0085	49.63 $\pm$ 3.13
Hybrid adaptive	61.45 $\pm$ 0.48	0.2834 $\pm$ 0.0042	50.35 $\pm$ 5.66

1. **Fixed** $k = 15$.
2. **Global cosine annealing** – uniform allocation across layers; 45 $\rightarrow$ 5 on Peptides and 30 $\rightarrow$ 10 on PascalVOC-SP.
3. **Layerwise cosine annealing** – the same temporal schedule with depth-dependent scaling from Eq. (2).
4. **Hybrid entropy-adaptive** – layerwise cosine plus per-query allocation from Eqs. (3)–(4).

Figure 1 illustrates these four schedule families directly. It is included because the schedules themselves are part of the method: the difference between a fixed budget, a global curriculum, a depth-dependent curriculum, and a query-adaptive refinement is easier to see visually than from the equations alone.

3 Results

3.1 Main predictive results

Table 1 summarises the final three-seed test results. The paper’s GPS+ k -MIP row is kept only as context. The direct comparison of interest is among the four methods implemented here.

The final pattern differs substantially from the earlier single-run exploration and is much more cautious. Under the final 50/60-epoch training budget, the **fixed baseline is strongest on both peptide benchmarks**. On **Peptides-func**, fixed $k = 15$ reaches 63.01 ± 0.67 AP, compared with 61.14 ± 1.67 for global cosine, 60.77 ± 0.74 for layerwise cosine, and 61.45 ± 0.48 for the hybrid method. On **Peptides-struct**, fixed $k = 15$ again performs best, with 0.2652 ± 0.0039 MAE versus 0.2784 ± 0.0056 (cosine), 0.2812 ± 0.0085 (layerwise), and 0.2834 ± 0.0042 (hybrid). In contrast, **PascalVOC-SP clearly benefits from adaptation** with the hybrid schedule reaching 50.35 ± 5.66 F1, compared with 49.63 ± 3.13 for layerwise cosine, 47.99 ± 0.97 for global cosine, and 47.23 ± 7.53 for fixed k .

Figure 2 adds a useful complementary view to these three-seed averages. Whereas Table 1 is the main evidential summary, the absolute-score bar chart shows the same methods against the paper reference values in one representative run. The key derivation from this figure is to show how strongly PascalVOC-SP can benefit in an individual run from adaptive scheduling, in contrast to the more stable peptide ordering, making it easier to establish the dataset dependence claim visually. The peptide tasks remain broadly aligned with the fixed- k ranking, whereas PascalVOC-SP has more

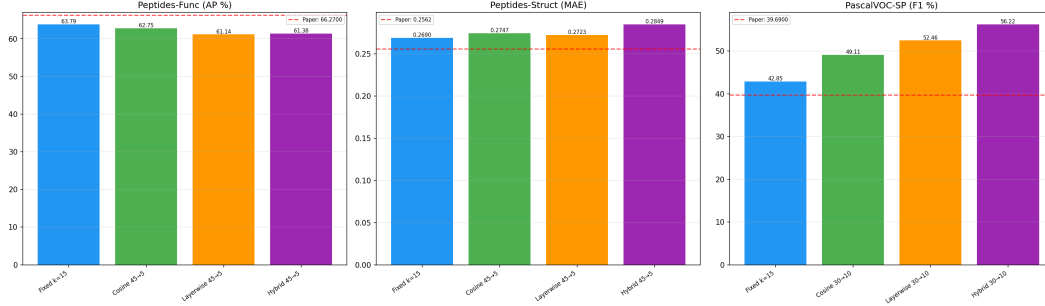


Figure 2: Single representative-run absolute-score comparison against the paper reference values. This figure complements Table 1 by showing the same ranking in absolute terms at run level rather than only through three-seed aggregates. PascalVOC-SP exhibit a much larger one-run upside for adaptive scheduling than the three-seed mean suggests, while the peptide ranking remains qualitatively unchanged.

room for adaptive gains.

This means that the final evidence does *not* support a universal benefit from adaptive sparsification. Instead, it supports a more specific claim, i.e., under the available compute budget, fixed k remains strongest on the smaller graph-level peptide tasks, whereas adaptive scheduling is clearly useful on the larger, more structurally diverse node-classification benchmark. This distinction is reinforced by the early-stopping statistics in the three-seed summary. On **Peptides-func**, the adaptive methods typically stopped after about 45–48 epochs, whereas fixed k always ran the full 50 epochs. On **Peptides-struct**, the adaptive methods often stopped much earlier, around 30–31 epochs on average, while fixed k again used the full budget. PascalVOC-SP behaved differently, the adaptive methods typically trained for about 47–53 epochs, whereas the fixed baseline stopped after only 37.3 ± 19.6 epochs on average. The adaptive schedules did not simply lose on the peptide datasets because they were undertrained, but they also did not need extra epochs there; on PascalVOC-SP, by contrast, they tended to keep improving for longer.

All headline predictive comparisons in this section come from the three-seed aggregates in Table 1 together with the absolute-score and delta views in Figures 2 and 3. Figures 4 and 5 instead use a single representative run to expose within-run phenomena that are not preserved by seed aggregation, namely entropy-collapse pace and the evolution of depth-dependent attention budgets. Figure 6 keeps the complementary cost summary.

3.2 Ablation of adaptive components

The four methods can be read as a component ablation of three adaptive ideas: temporal annealing, depthwise allocation, and querywise entropy adaptation. Comparing **fixed k** with **global cosine** isolates the effect of temporal annealing alone. Comparing **global cosine** with **layerwise cosine** isolates the extra effect of redistributing the budget across depth. Comparing **layerwise cosine** with **hybrid** isolates the contribution of entropy-based query adaptation.

Temporal annealing alone hurts both peptide tasks relative to fixed k (-1.88 AP on Peptides-func and $+0.0132$ MAE on Peptides-struct, where higher MAE is worse), but gives a small gain on PascalVOC-SP ($+0.76$ F1). **Layerwise allocation** still remains below fixed k on both peptide datasets, but it improves over plain cosine on PascalVOC-SP (49.63 ± 3.13 vs. 47.99 ± 0.97 F1). **Entropy adaptation** is mixed: it recovers some AP on Peptides-func relative to layerwise cosine, slightly worsens Peptides-struct further, and gives the best overall PascalVOC-SP result, improving fixed k by about $+3.13$ F1. The negative result is therefore informative in itself: adding more adaptive machinery is not automatically beneficial, and the full hybrid design is useful here only on PascalVOC-SP.

Figure 3 shows the same conclusion graphically using the three-seed mean differences relative to fixed $k = 15$.

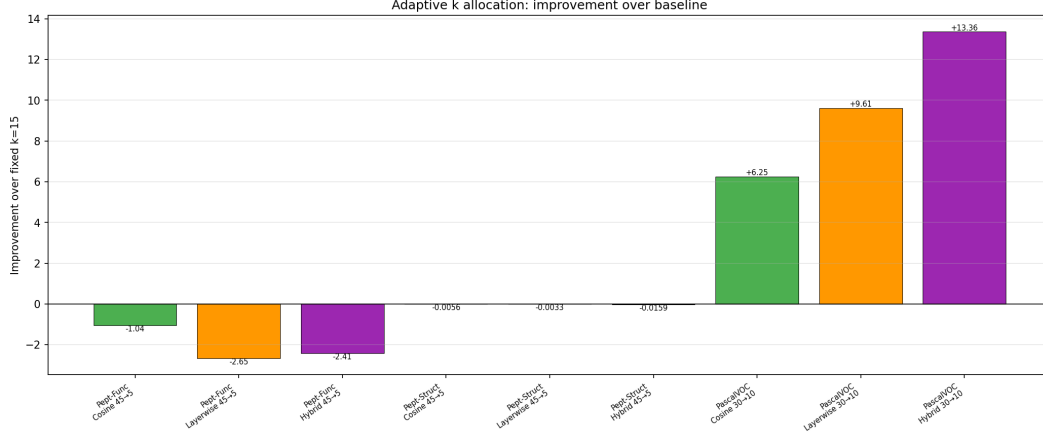


Figure 3: Improvement over the fixed- k baseline using three-seed means. For MAE, the sign is flipped so that positive means better. The final study shows negative deltas on both peptide datasets and substantial positive deltas on PascalVOC-SP.

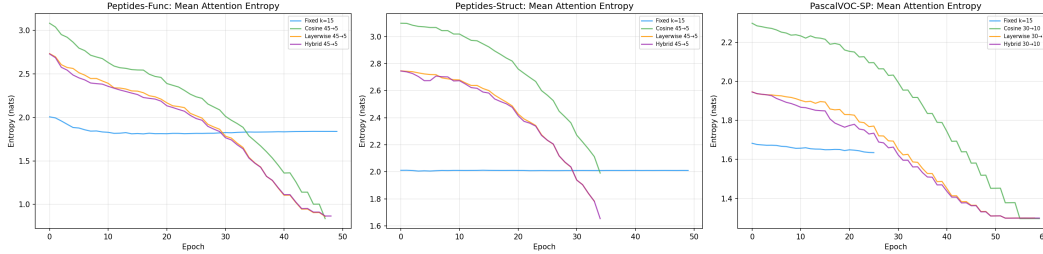


Figure 4: Representative-run mean attention entropy across epochs. The adaptive schedules all drive entropy downward relative to fixed k , confirming that annealing produces increasingly concentrated attention distributions. The schedules clearly change attention behaviour even on datasets where the final score does not improve.

3.3 Entropy evolution and layerwise schedule behaviour

Figure 4 provides the first mechanism check from a representative run. Mean attention entropy falls steadily for all adaptive schedules, while the fixed baseline remains comparatively flat and annealing does indeed make attention progressively more selective over training. On Peptides-func and PascalVOC-SP, the cosine, layerwise, and hybrid curves all move well below the fixed- k line by late epochs, confirming that the schedules are narrowing the effective search space. On Peptides-struct, however, a sharper entropy collapse does not translate into a better final MAE, which reinforces the paper’s central point that changing the sparsity pattern and improving downstream prediction are not the same thing.

Figure 5 provides the second mechanism check. The per-layer $k_{\max}$ profiles show that the layerwise and hybrid schedules preserve the largest budgets in shallow layers and reduce them more aggressively in deeper layers confirming the intended depth-aware behaviour outlined in Eq. (2). The PascalVOC-SP panel’s 30→10 schedule decays more gently than the peptide 45→5 schedule, making it visually clear that the final study did not use one universal endpoint range across datasets.

3.4 Runtime and memory trade-offs

The cost picture is again dataset-dependent. On the peptide benchmarks, adaptive schedules often *reduced total wall time* relative to fixed k because they stopped earlier, but they consistently used *more peak memory*. For example, on Peptides-Struct the fixed baseline took 1643.3 ± 10.0 s on average, whereas cosine and layerwise took 1053.2 ± 146.2 s and 1061.7 ± 156.6 s respectively. However, fixed k used only 2122.8 ± 6.6 MB peak memory, compared with 3013.0 ± 7.5 MB for

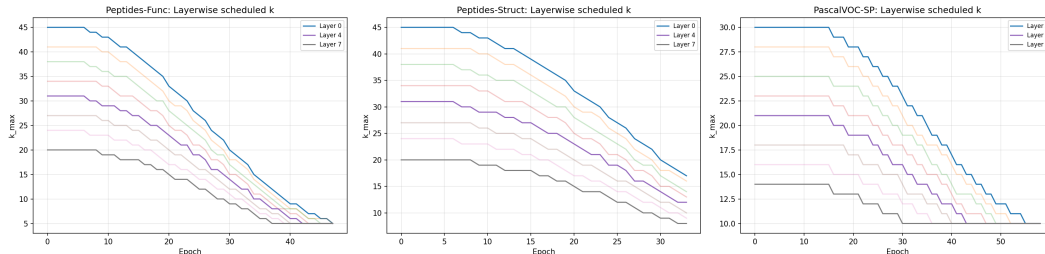


Figure 5: Representative-run layerwise $k_{\max}$ profiles across epochs. Earlier layers retain broader attention budgets throughout training, while deeper layers are forced into tighter sparsity earlier. PascalVOC-SP uses a visibly gentler 30→10 decay than the peptide 45→5 schedule.

cosine and 2633.6 ± 7.5 MB for layerwise. On PascalVOC-SP, the pattern reverses: fixed k is both the cheapest in time and the lowest-memory option, while the adaptive schedules cost more but buy substantially better F1. Because this figure is secondary to the predictive and mechanism results, the full runtime/memory plot is moved to Appendix A as Figure 6. It should be interpreted as an *overall training-cost analysis under the chosen stopping policy*, not as a pure per-epoch algorithmic-efficiency comparison.

4 Discussion and Critical Perspective

4.1 What the final study supports

Adaptive k -allocation is **not** a robust win across all datasets. Instead, its benefit appears most clearly on the larger, more structurally diverse PascalVOC-SP benchmark, where all adaptive methods outperform the fixed baseline and the full hybrid design performs best. On the smaller peptide benchmarks, by contrast, fixed $k = 15$ remains the strongest option under the final compute budget. A reasonable reading is that larger and more heterogeneous graphs have more to gain from early exploration and from query- or depth-specific sparsity control, whereas the smaller peptide graphs may already be well served by a moderate fixed budget.

4.2 Relation to course concepts

This study operationalises several core course ideas.

Permutation symmetry and the GDL blueprint. The graph lectures framed graph learning through permutation invariance and equivariance: nodewise updates must respect arbitrary node orderings, while graph-level readouts must be permutation-invariant. GraphGPS fits that blueprint directly. Its local GatedGCN block is a standard permutation-equivariant message-passing layer, and the final graph prediction is permutation-invariant after pooling. The adaptive k schedules do not change that symmetry class; they change only how much interaction is allowed through the global channel.

Hybrid MPNN–Transformer modelling. The course presented transformers on graphs as attentional GNNs on a complete graph, where attention weights act as a learned adjacency. GraphGPS instantiates exactly this hybrid view by combining local message passing with a global attention block. In this report, adaptive k makes that lecture idea concrete: it controls how much of the learned long-range interaction graph is available at different stages of training and at different depths.

Long-range interactions and oversquashing. The graph lectures emphasised that purely local message passing can struggle when tasks require distant information to mix across bottlenecks. The adaptive schedules tested here are a practical response to that issue. Larger early budgets widen the route for long-range interaction before the model commits to sharper sparsity, while depth-dependent schedules preserve broader interaction in earlier layers and enforce stronger selectivity only after useful global structure has been identified.

Relation to the practicals. The practicals made these ideas concrete at model level. Practical 1 distinguished learned robustness from architectural inductive bias through invariance and augmentation; that same distinction appears here in the contrast between fixed sparsity and scheduled sparsity.

Practical 2 required implementing a node-classification GNN from scratch, which makes the local message-passing component of GraphGPS conceptually familiar. Practical 4 returned to invariant and equivariant graph models, so the present report extends that thread from symmetry-preserving message passing to hybrid architectures that combine local graph structure with learned global attention.

4.3 Critical self-evaluation and limitations

Although the final results use three seeds, this is still a small sample and the PascalVOC-SP standard deviations remain large. The conclusions are explicitly **compute-constrained**: the final budgets are 50 epochs for Peptides and 60 epochs for PascalVOC-SP, so the study measures performance under a fixed training budget rather than fully convergence-matched performance. This matters especially for adaptive schedules, which may need longer to realise their benefit. The paper comparison remains imperfect because our parameter budget is smaller and PascalVOC-SP uses a custom split. The mechanism figures are representative-run diagnostics rather than seed-averaged statistics, and they are used only to expose convergence, entropy-collapse, and layerwise-budget dynamics that are not preserved in the three-seed tables. Finally, the study does not include the larger targeted ablation over schedule range, warmup, and patience that would be needed to isolate those hyperparameters formally.

That said, the present design is still informative. It provides a component ablation of adaptive ideas, a three-seed comparison, and a cost-performance analysis. A cautious practical takeaway is could therefore be, for small graph-level tasks under limited compute, fixed k is a strong default. For larger or more structurally diverse graphs, adaptive schedules, with the hybrid variant performing best on PascalVOC-SP in this study, deserve serious consideration. The rule-of-thumb from Eq. (5), namely to keep final coverage at a few percent of average graph size, remains a useful starting point rather than a validated law.

Several follow-up studies are needed before drawing stronger conclusions. The most immediate next step is a larger multi-seed evaluation with confidence intervals and significance testing. Beyond that, it would be valuable to measure convergence under larger epoch budgets, to run a dedicated ablation over range and warmup, and to test the schedules on additional graph families with wider size variation. These extensions would turn the present study from a careful compute-constrained comparison into a more conclusive account of when adaptive k is worth using.

References

- [1] J. De Schouwer, H. Sáez de Ocáriz Borde, and X. Dong, “k-Maximum Inner Product attention for graph transformers and the expressive power of GraphGPS,” *arXiv preprint arXiv:2604.03815*, 2026.
- [2] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, “Recipe for a general, powerful, scalable graph transformer,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017.
- [4] B. Charlier, J. Feydy, J. A. Glaunès, F.-D. Collin, and G. Durif, “Kernel operations on the GPU, with autodiff, without memory overflows,” *Journal of Machine Learning Research*, vol. 22, no. 74, pp. 1–6, 2021.
- [5] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th International Conference on Machine Learning*, 2009, pp. 41–48.
- [6] V. P. Dwivedi, L. Rampášek, M. Galkin, A. Parviz, G. Wolf, A. T. Luu, and D. Beaini, “Long range graph benchmark,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [7] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.

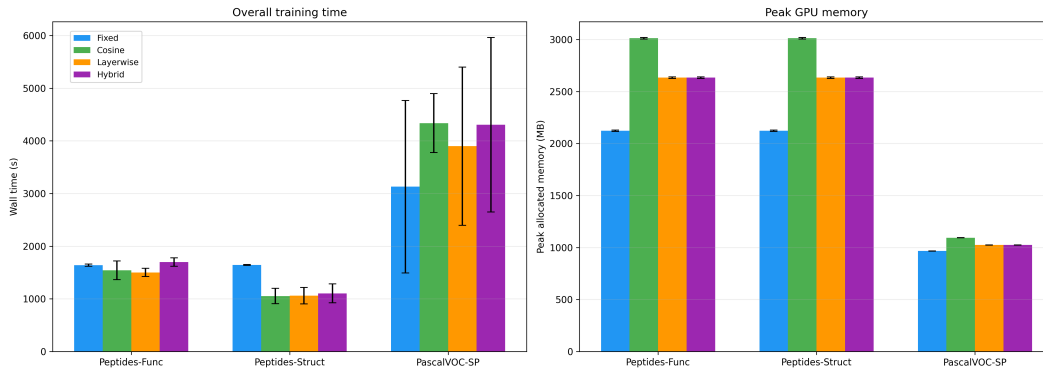


Figure 6: Overall training time and peak allocated GPU memory (three-seed mean $\pm$ std). Bars are grouped by method family (Fixed, Cosine, Layerwise, Hybrid). The practical lesson from this appendix figure is a cost–performance trade-off: adaptive schedules often stop earlier on the peptide datasets but use more peak memory, while on PascalVOC-SP they cost more overall yet buy substantially better F1.

A Supplementary figures

The appendix figure collects practical cost information that is secondary to the predictive and mechanism results in the main text.